1. Considerati instructiunile de mai jos si descrieti continutul fiecarei variablile la executia acestora cand n are valoarea 3
   Consider the instructions below, describe the content of each variable while executing the program when n is 3.

```
x <- n
for k <- 0 to n-1 do
        j <- 0
        while j <= k do
                x <- x + x
                j <- j + 1
```

We will create a table, who will volunteer?

Step	Instruction	Memory
1		
2		
3		
4		

2. Sa se determine numărul de operaţii efectuate de către un algoritm care determina **numărul** de inversiuni ale unei permutări.
   Determine the number of operations performed by an algorithm that determines the number of inversions of a permutation.

Pentru o permutare (a1, . . . , an), a.i. i∈ {1, . . . , n}, i = 1, n o inversiune este o pereche de indici (i, j) cu proprietatea ca i < j si $a_i > a_j$.

For a permutation (a1, . . . , an), a.i. i∈ {1, . . . , n}, i = 1, n; an inversion is a pair of indices (i,j) with the property that i<j and $a_i > a_j$.

```
procedure inverse(a[1,..,n], n):
    k = 0
    for i = 1; i < = n-1; i++ do
        for j = i + 1 ; j <= n ; j++ do
            if a[i] > a[j] then
                k = k + 1
    return k
end
```

Index	Cost	Execution times
1	1	1
2	2(n-1+1)	1
3	2(n-i+1)	i = 1,n-1
4	1	i = 1,n-1; j = i+1,n
5	τ	i = 1,n-1; j = i+1,n

$$T(n) = 1 + 2n + 2\sum_{i=1}^{n-1}(n-i+1) + \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} 1 + \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} \tau = \frac{3n(n-1)}{2} + 4n - 1 + \sum_{i=1}^{n-1}\sum_{j=i+1}^{n} \tau$$

deci

$$\frac{3n^2+5n-2}{2} \leq T(n) \leq 2n^2 + 2n - 1.$$

3. Precizati si demonstrati complexitatea timp a următorului algoritm.
   Determine and specify the time complexity of the following algorithm:

```
sum <- 0
for i <- 0 to n-1 do
    for j <- i to n-1 do
        for k <- 0 to 4 do
            sum <- sum + 2
```

Index	Cost	Repetitions
1	1	1
2	2(n+1)	1
3	2(n-i+1)	i=0,n-1
4	2*5	i=0,n-1, j=i,n
5	2	i=0,n-1, j=i,n, k=0,4

$SUM^{n-1}_{i=0}$ is a sum from i=0 till n-1

$T(n) = 1 + 2 * (n + 1) + 2*SUM^{n-1}_{i=0}(n-i+1) + 2*SUM^{n-1}_{i=0} SUM^{n}_{j=i}(5) +SUM^{n-1}_{i=0} SUM^{n}_{j=i}SUM^{4}_{k=0}(2)$

After this we need to calculate...

$$T_a(n) = \sum_{i=0}^{n-1}\left(\sum_{j=i}^{n-1}\left(\underbrace{\sum_{k=0}^{4} 1}_{5}\right)\right) = \sum_{i=0}^{n-1}\left(\sum_{j=i}^{n-1} 5\right) = \sum_{i=0}^{n-1} 5(n-i) = 5 \cdot \frac{n(n+1)}{2} \Rightarrow T_a(n) = \frac{5}{2}n^2 + \frac{5}{2}n \Rightarrow$$

$$\Rightarrow T_a(n) \in O(n^2$$

4. Sa se determine termenul dominant si ordinul de crestere pentru expresiile:
   Determine the dominant term and the growth order for the expressions:

- T(n)=2lgn + 4n^n + 3nlgn

The dominant term in 2lgn + 4n + 3nlgn is 3nlgn and the growth order is n log n.

- 2 + 4 + 6 + . . . 2n - is it 2n?

Dominant term of the sum 2 + 4 + 6 + . . . 2n is not 2n but n^2

2(1+2+..+n) = 2(n(n+1))/2 = n^2 + n

Dominant term is n^2 and the growth order is n^2.

5. **Prove that: $n! \in O(n^n)$**

n! < = n ^ n

Deci, in limbaj natural, sa se demonstreze ca n! este MARGINIT SUPERIOR de n^n.

In natural language we must prove that n! is bounded by n^n, n^n is an upper bound.

Do not forget O is the notation that means: “worse case scenario of an algorithm”

We must demonstrate that n! < = n ^ n

----------Base case----------
P(1) : 1! < = 1 ^ 1 ? Yes
P(2): 2! < = 2 ^ 2 ? 2 < = 4 ? Yes

----------Induction step----------
P(3): Say that: 3! = 3 * 2! write (k + 1) ! = (k + 1) * P(k) - current step * last step

3 * P(2) < = 3 ^ 3 (Yes, 6 < = 27)
.
.
.
Suppose that P(k) is true, which means k! < = k ^ k and we will prove P(k+1)

Prove P(k+1) is true:

P(k+1): (k+1)! = (k + 1) * (k) * (k-1) … (3)*(2)*(1)
= (k + 1) * k! “- P(k)”
⇒ using the upper supposition that (k! < = k ^ k)

It will result that (k+1) * k! <= (k+1)* k ^ k

Knowing that k < k +1 din (k+1) * k! < (k+1)* k ^ k it will result that

(k+1) * k! <= (k+1)* k ^ k <= (k+1)(k+1) ^ k
= (k+1)^(k+1)

---

or calculate

lim n!/n^n
n-> infinity

limita e 0, daca stam sa calculam, asta inseamna ca n! creste mai greu decat n^n => n^n il margineste superior => am demonstrat

limit is 0 if we calculate, that means that n! grows slower than n^n hence it is a superior bound, hence we proved.

Growth orders

In order to compare the growth orders of two running times $T1(n)$ and $T2(n)$, the $lim_{n\to\infty}\frac{T1(n)}{T2(n)}$ is computed.

- if $lim = 0$: $T1(n)$ has a smaller growth order than $T2(n)$;
- if $lim = c, c > 0$ constant: $T1(n)$ and $T2(n)$ have the same growth order;
- if $lim = \infty$: $T1(n)$ has a bigger growth order than $T2(n)$.